PUC Minas



Algoritmos e Estruturas de Dados I

Prof. Dr. João Paulo Aramuni

Aula 03

Abstrações de controle

AEDS I - Manhã

03

Abstrações de controle

Unidade de Ensino

3.1 Noções de modularização

3.2 Funções e procedimentos

3.3 Noções de endereçamento e alocação de memória

3.4 Passagem de parâmetros por valor e por referência

3.5 Recursividade



3.1 Noções de modularização

No contexto do desenvolvimento, "**modularização**" refere-se à prática de dividir um programa em módulos ou unidades independentes, cada uma realizando uma função específica.

Essa abordagem ajuda a organizar o código, facilita a manutenção, melhora a legibilidade e reutilização do código.

Vejamos algumas noções importantes relacionadas à modularização.



3.1 Noções de modularização

Divisão em módulos:

- Consiste em dividir um programa em partes menores, chamadas módulos. Cada módulo deve ter uma responsabilidade específica e realizar uma tarefa bem definida.

Encapsulamento:

- Os detalhes internos de um módulo são ocultos para outros módulos, promovendo a independência e reduzindo o acoplamento entre partes do código.



3.1 Noções de modularização

Reutilização de código:

- A modularização permite que módulos específicos sejam reutilizados em diferentes partes do programa ou em diferentes projetos.

Facilidade de manutenção:

- Cada módulo pode ser mantido separadamente, facilitando a identificação e correção de erros ou a introdução de melhorias.



3.1 Noções de modularização

Legibilidade do código:

- A estrutura modular torna o código mais legível, pois os desenvolvedores podem se concentrar em entender uma parte específica do programa de cada vez.

Interfaces claras:

- Os módulos devem ter interfaces bem definidas, especificando como outros módulos podem interagir com eles.



3.1 Noções de modularização

Facilidade de teste:

- Módulos independentes podem ser testados individualmente, simplificando o processo de identificação de problemas e a realização de testes unitários.

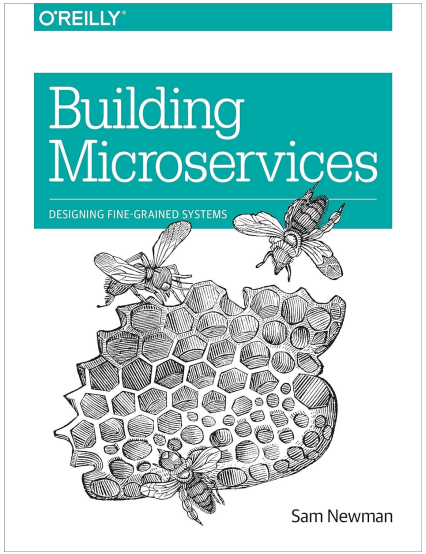
Ao aplicar essas noções de modularização, os programadores conseguem criar sistemas mais robustos, flexíveis e fáceis de entender e manter. Isso é especialmente importante em projetos de grande porte, nos quais a complexidade pode ser significativa.



3.1 Noções de modularização

Leituras sugeridas:





3.2 Funções e procedimentos

Funções e procedimentos são blocos de código que realizam tarefas específicas.

Ambos são usados para organizar e estruturar o código, facilitando a modularização e reutilização do código. No entanto, há uma diferença conceitual entre funções e procedimentos.



3.2 Funções e procedimentos

Funções:

As funções em C são blocos de código que executam uma tarefa específica e podem retornar um valor para o chamador.

Elas são definidas usando o tipo de dado que a função retornará (como int, float, etc.) ou void se a função não retornar nenhum valor.



3.2 Funções e procedimentos

Um exemplo de uma função em C:

```
int somar(int a, int b) {
    return a + b;
}
```

Neste exemplo, a função somar aceita dois argumentos inteiros, a e b, e retorna a soma deles.



3.2 Funções e procedimentos

Protótipo de função:

O protótipo de função em C é uma declaração antecipada da função antes de ser definida. Ele fornece ao compilador as informações necessárias sobre a função, como seu nome, tipos de parâmetros e tipo de retorno, antes de encontrar a definição real da função.

Isso é útil porque o C é uma linguagem de compilação única, o que significa que o código é compilado em uma única passagem.

3.2 Funções e procedimentos

Protótipo de função:

A inclusão do protótipo de função no início do código permite que o compilador saiba o que esperar mais adiante no código, permitindo que ele verifique se as chamadas de função estão corretas em termos de número e tipos de argumentos, bem como o tipo de retorno.

Ao usar o protótipo, você pode separar a declaração da função da sua implementação.

3.2 Funções e procedimentos

Protótipo de função:

Isso é útil em programas grandes, onde várias funções podem ser definidas em diferentes arquivos.

Colocar os protótipos em um cabeçalho (header) permite que outros arquivos fonte saibam como usar essas funções sem precisar conhecer todos os detalhes internos.

3.2 Funções e procedimentos

```c
#include <stdio.h>
#include <stdlib.h>

/**
 * Protótipo da função (assinatura)
 *
 * Esta função recebe um número como argumento e retorna o quadrado desse número.
 *
 * @param numero (int) - O número para o qual queremos calcular o quadrado.
 * @return int - O quadrado do número de entrada.
 */
int calcularQuadrado(int numero);

int main(void) {
    // Exemplo de uso da função
    int numeroEntrada = 5;

    // Chamando a função e armazenando o resultado
    int resultado = calcularQuadrado(numeroEntrada);

    // Exibindo o resultado
    printf("O quadrado de %d é: %d\n", numeroEntrada, resultado);

    return 0;
}
```

3.2 Funções e procedimentos

Com a assinatura da função no início do programa, podemos definir a função ao final do código, sem problemas.

```
/**
 * Definição da função calcularQuadrado
 *
 * Esta função recebe um número como argumento e retorna o quadrado desse número.
 *
 * @param numero (int) - O número para o qual queremos calcular o quadrado.
 * @return int - O quadrado do número de entrada.
 */
int calcularQuadrado(int numero) {
    // Calculando o quadrado do número
    int quadrado = numero * numero;

    // Retornando o resultado
    return quadrado;
}
```

3.2 Funções e procedimentos

Veja o que acontece quando definimos a função ao final do código, mas sem assiná-la no início do programa:

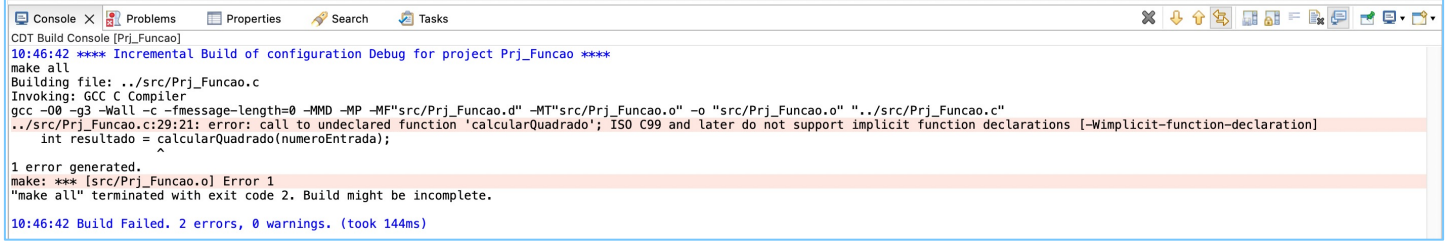
```c
// int calcularQuadrado(int numero);

int main(void) {
    // Exemplo de uso da função
    int numeroEntrada = 5;

    // Chamando a função e armazenando o resultado
    int resultado = calcularQuadrado(numeroEntrada);

    // Exibindo o resultado
    printf("O quadrado de %d é: %d\n", numeroEntrada, resultado);

    return 0;
}
```



3.2 Funções e procedimentos

Para corrigir esse erro:

I) Ou assinamos a função no início do programa;

II) Ou movemos a função inteira para o início do programa.

O protótipo de função em C ajuda a organizar o código, facilita a leitura e manutenção, e permite ao compilador verificar a consistência das chamadas de função em todo o programa.

3.2 Funções e procedimentos

Procedimentos:

Procedimentos em C são semelhantes às funções, mas não retornam um valor ao chamador.

Eles realizam uma tarefa ou conjunto de tarefas sem a necessidade de retornar um valor. Geralmente, os procedimentos são declarados usando o tipo de dado void.



3.2 Funções e procedimentos

Um exemplo de um procedimento em C:

```
void imprimirMensagem() {
    printf("Olá, mundo!\n");
}
```

Neste exemplo, o procedimento imprimirMensagem não retorna nenhum valor, apenas imprime uma mensagem na tela.



3.2 Funções e procedimentos

Diferenças principais:

Funções podem retornar valores, enquanto procedimentos não retornam nada (ou retornam void).

- Ao chamar uma função, você geralmente espera usar o valor retornado em algum lugar. Já com procedimentos, você os chama para realizar uma tarefa específica, sem se preocupar com um retorno. Ambos são usados para modularizar o código e facilitar a leitura, mas a escolha entre função e procedimento depende se você precisa ou não de um valor de retorno.

3.3 Noções de endereçamento e alocação de memória

Em linguagem C, o termo "**endereçamento**" refere-se à manipulação de endereços de memória. Os endereços de memória são números que identificam a localização de dados na memória do computador.

A manipulação de endereços é uma parte essencial da programação em C, pois permite que você acesse e manipule dados armazenados em diferentes locais da memória.



3.3 Noções de endereçamento e alocação de memória

Em C, as variáveis são armazenadas em locais específicos da memória, e o endereço de uma variável é o local onde ela está armazenada.

Você pode usar operadores de endereço, como o operador **&**, para obter o endereço de uma variável.



3.3 Noções de endereçamento e alocação de memória

Por exemplo:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int minhaVariavel = 13;
    printf("O endereço de memória de 'minhaVariavel' é: %p\n", &minhaVariavel);
    return 0;
}
```

Console × Problems Properties Search Tasks

<terminated> (exit value: 0) Prj_Memoria [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_Memoria/Debug/Prj_Memoria (13/02/2024 09:50)

```
O endereço de memória de 'minhaVariavel' é: 0x16b746c18
```

3.3 Noções de endereçamento e alocação de memória

O especificador de formato **%p** é usado em funções de impressão, como printf em C, para imprimir ponteiros em formato hexadecimal.

Um **ponteiro** é uma variável em programação que armazena o endereço de memória de outra variável. Em outras palavras, um ponteiro contém o endereço de uma localização específica na memória onde dados podem ser armazenados.

A principal utilidade dos ponteiros está em permitir a manipulação indireta de dados, o que oferece maior flexibilidade e eficiência em certos contextos.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática**

Na alocação estática de memória, o programador não precisa se preocupar em reservar memória para seus dados.

A quantidade de memória necessária para armazenar suas variáveis é automaticamente reservada na **stack** (pilha) ou em outras seções do programa. Do mesmo modo, o programador também não tem controle sobre o tempo de vida dessas variáveis na memória.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática**

A **stack** guarda os dados alocados dentro dos escopos de funções: variáveis locais, parâmetros, retorno de funções e endereços de outras áreas do código.

Nela, as instruções e os dados vão sendo empilhados, e o desempilhamento ocorre automaticamente após a execução.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática**

Nesse tipo de alocação os dados estão organizados sequencialmente na memória do computador (como os arrays).

Além disso, a quantidade total de memória utilizada pelo programa é previamente conhecida e não pode ser mudada (não é possível redimensionar a memória).

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática**

As variáveis são alocadas considerando toda a memória que seu tipo de dado necessita e não consideram a quantidade que seria realmente necessária na execução do programa.

Consequentemente, podemos ter espaços alocados na memória que não são utilizados.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática**

A figura a seguir mostra um exemplo passo a passo do funcionamento da **stack**. As variáveis são empilhadas na stack à medida que são declaradas na main() (Passos 1 e 2).

Ao correr uma chamada de função, cria-se uma nova região na stack com os parâmetros locais (Passo 3).

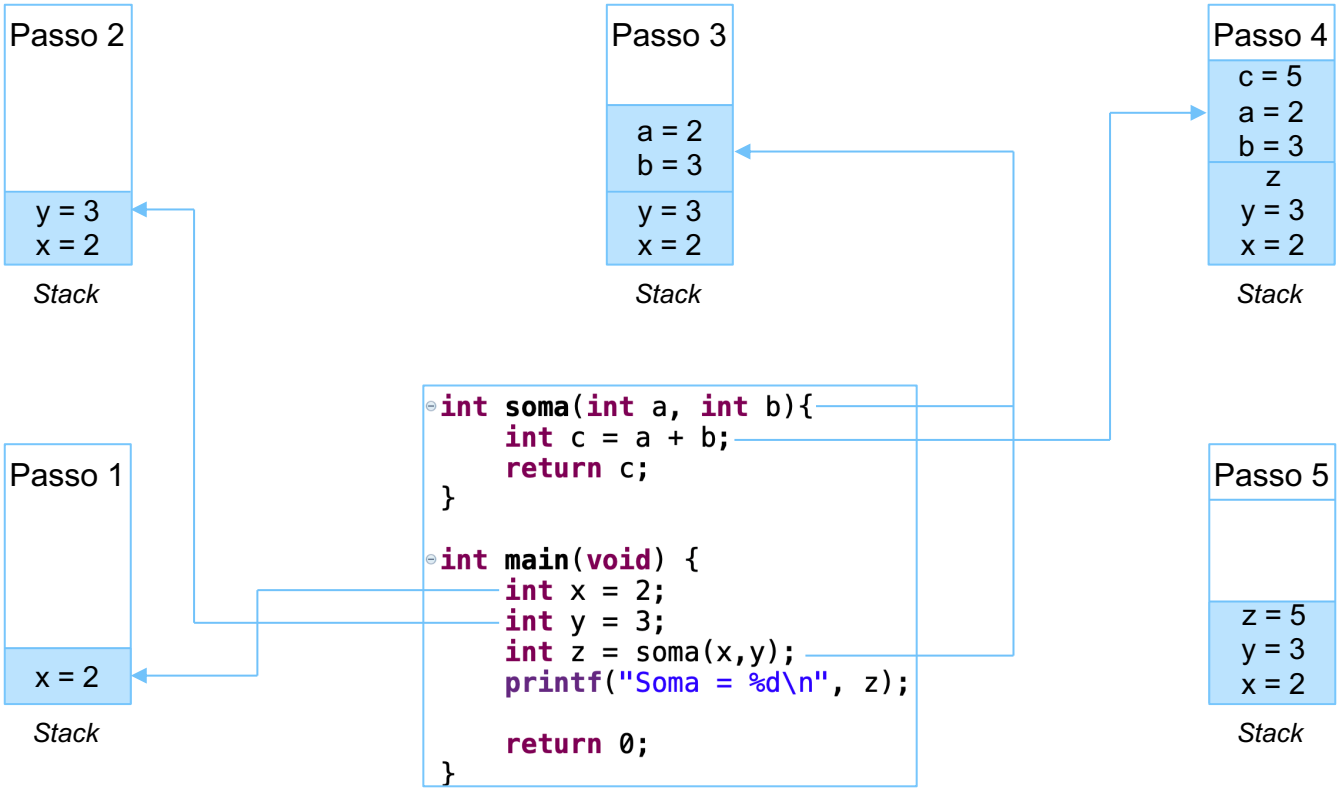
3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática**

Variáveis declaradas dentro da função são empilhadas dentro da sua região da stack (Passo 4). Terminada a execução da função ela é removida da stack (desempilhada) e seu valor retornado para uma variável dentro da main().

A variável declarada para receber o retorno da função é empilhada dentro da região da stack associada a main() (Passo 5).

3.3 Noções de endereçamento e alocação de memória



3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

Na alocação dinâmica de memória, o programador tem total controle sobre o tamanho e o tempo de vida das posições de memória dos seus dados.

Ou seja, o programador é responsável por reservar a quantidade de memória necessária para seus dados. Essa memória não é mais reservada na stack, mas em outra área de memória chamada **heap**.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

As variáveis armazenadas na heap não dependem do escopo e devem ser liberadas manualmente pelo programador.

Se o programador esquecer de liberar essa memória, um vazamento de memória (memory leak) poderá ocorrer, causando falha no aplicativo.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

Heap é uma seção do programa que é bem maior que a **stack**. Ela é ideal para alocar muitos objetos e/ou objetos muito grandes. Nela, os dados são alocados dinamicamente por meio da função **malloc()** em C e só podem ser acessados por ponteiros.

Além de ser mais lenta que a stack, a remoção de objetos da heap é manual.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

Nesse tipo de alocação os dados não precisam estar necessariamente organizados sequencialmente na memória do computador.

Em geral, os blocos de memória alocados podem estar dispersos na memória do computador (estruturas encadeadas fazem melhor uso desse tipo de memória).

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

Além disso, a quantidade total de memória utilizada pelo programa não precisa ser previamente conhecida. Os blocos de memória podem ser alocados, liberados e realocados para diferentes propósitos, durante a execução do programa.

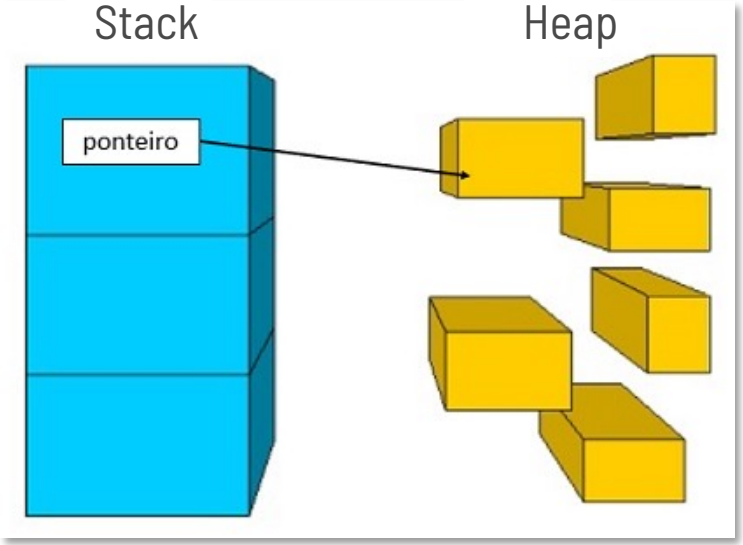
O acesso aos dados agora é feito por ponteiros que apontam para os blocos de memória alocados, e não mais por variáveis.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

Stack - Uma área de tamanho médio de 1 MB a 4 MB;

Heap - Uma área que pode ter Gigabytes de tamanho.



3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

A figura a seguir mostra um exemplo passo a passo do funcionamento da **heap**. As variáveis são empilhadas na stack à medida que são declaradas na main() (Passo 1).

Ao correr uma chamada da função **malloc()**, cria-se uma nova região de memória na heap e o ponteiro que aponta para essa região é empilhado na stack (Passos 2 e 3).

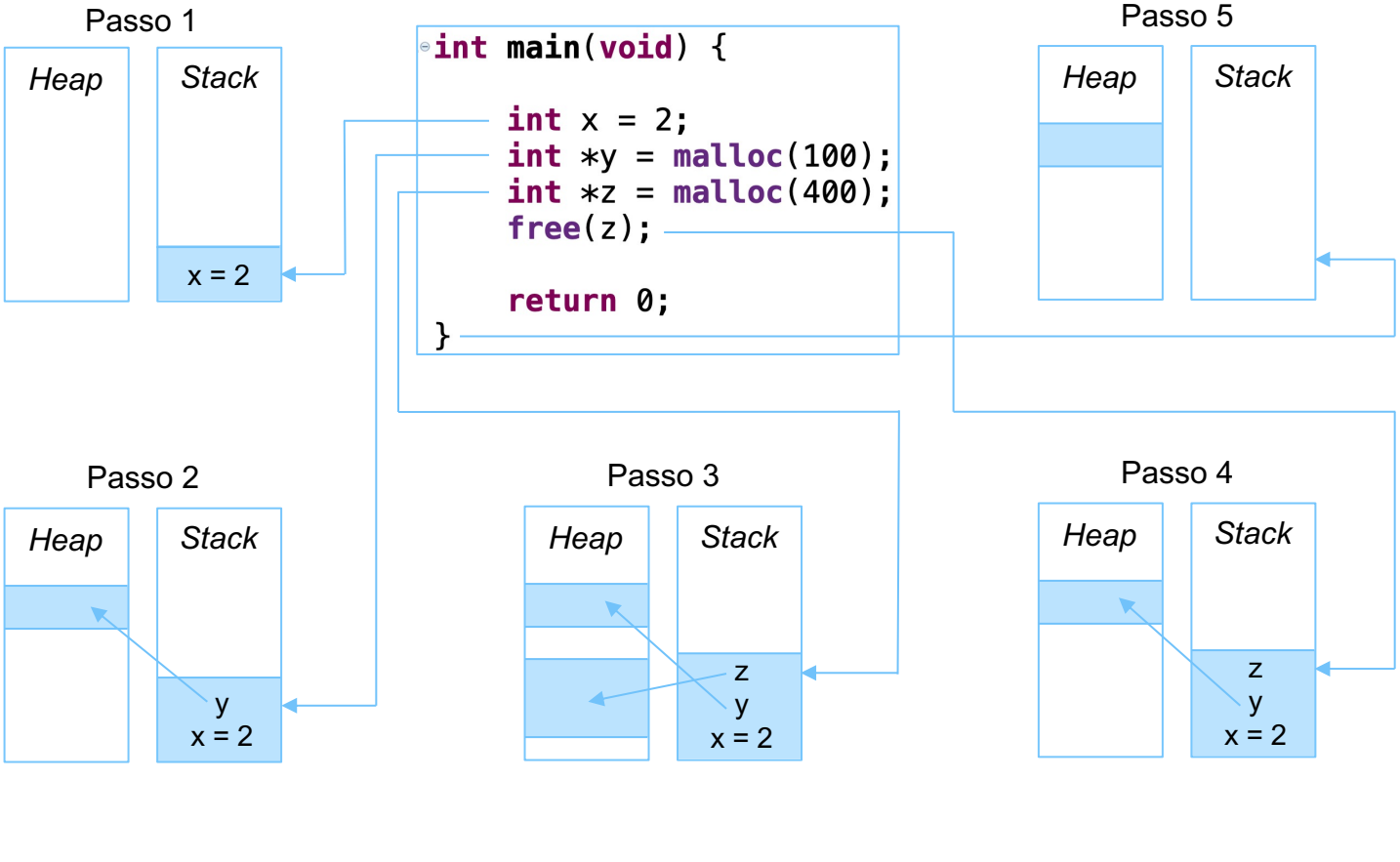
3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Dinâmica**

Note que as regiões na heap não são escolhidas de forma sequencial. Quando a função **free()** é chamada, a região associada a aquele ponteiro é liberada da heap, mas o ponteiro ainda continua na **stack** (Passo 4).

Ao fim da main(), todas as variáveis são removidas da stack. Porém, uma região da **heap** continua presente, pois ela foi alocada, mas nunca liberada (Passo 5).

3.3 Noções de endereçamento e alocação de memória



3.3 Noções de endereçamento e alocação de memória

Linguagens de programação mais recentes, como Python, Java, Rust e Swift, adotaram estratégias de gerenciamento de memória mais avançadas, aliviando os programadores da responsabilidade direta de alocar e desalocar memória manualmente.

Essas linguagens geralmente incorporam técnicas de gerenciamento automático de memória, como coleta de lixo (garbage collection) ou referências fortes e fracas, que permitem detectar e liberar automaticamente a memória não utilizada.

3.3 Noções de endereçamento e alocação de memória

Essa abstração facilita o desenvolvimento de software, uma vez que reduz consideravelmente o risco de vazamentos de memória e erros relacionados à manipulação manual de alocação.

Além disso, o gerenciamento automático de memória contribui para um código mais seguro, eficiente e menos propenso a bugs relacionados à alocação inadequada de recursos, permitindo que os desenvolvedores concentrem-se mais na lógica do programa do que em detalhes de baixo nível.

3.3 Noções de endereçamento e alocação de memória

No entanto, é importante ressaltar que, apesar desses benefícios, o entendimento dos conceitos de gerenciamento de memória continua sendo valioso para desenvolvedores, especialmente ao lidar com linguagens de programação que oferecem mais controle direto sobre esse aspecto.

3.3 Noções de endereçamento e alocação de memória

Alocação de memória: **Estática vs Dinâmica**

Alocação estática (stack)	Alocação dinâmica (heap)
Armazenado na memória RAM.	Armazenado na memória RAM.
Variáveis são liberadas automaticamente no fim do escopo.	Variáveis não dependem do escopo (podem ser acessadas globalmente) e devem ser liberadas manualmente.
Alocação mais rápida que na ***heap***.	Alocação mais lenta que na ***stack***.
Implementado usando uma estrutura de dados do tipo pilha.	Blocos de dados são alocados sob demanda.
Armazena dados locais e endereços de retorno utilizados na passagem de parâmetros.	Pode sofrer fragmentação após sucessivas alocações e liberações de memória.
Os dados podem ser usados sem ponteiros.	Os dados são acessados por ponteiros.
Pode ocorrer estouro de pilha quando for muito usada.	A alocação pode falhar se muita memória for solicitada.
É usada quando se sabe exatamente o quanto de espaço será alocado antes do tempo de compilação e esse espaço não é muito grande.	É usada quando não se sabe exatamente o quanto de espaço será alocado antes do tempo de compilação ou esse espaço é muito grande.
Geralmente possui um tamanho máximo predeterminado quando o programa inicia.	Responsável por vazamentos de memória.

3.4 Passagem de parâmetros por valor e por referência

Em linguagens de programação, como C, a passagem de parâmetros pode ocorrer de duas maneiras principais: por valor (passagem por valor) e por referência (passagem por referência).

Vamos entender cada uma delas.



3.4 Passagem de parâmetros por valor e por referência

Passagem por valor:

Na passagem por valor, o valor real do argumento é copiado para o parâmetro da função.

Qualquer alteração feita dentro da função no valor do parâmetro não afeta o argumento original fora da função.

É o método padrão para tipos de dados simples, como inteiros, caracteres e floats em C.



3.4 Passagem de parâmetros por valor e por referência

Passagem por valor:

```c
#include <stdio.h>
#include <stdlib.h>

void incrementarPorValor(int x) {
    x++;
}

int main(void) {
    int numero = 5;
    incrementarPorValor(numero);
    printf("Valor original: %d\n", numero); // Saída: Valor original: 5

    return 0;
}
```

3.4 Passagem de parâmetros por valor e por referência

Passagem por referência (ou endereço):

Na passagem por referência, o endereço de memória do argumento é passado para o parâmetro da função.

Isso permite que a função acesse e modifique diretamente o valor no endereço de memória original.

Em C, isso é frequentemente implementado usando ponteiros.



3.4 Passagem de parâmetros por valor e por referência

Passagem por referência:

```c
#include <stdio.h>
#include <stdlib.h>

void incrementarPorReferencia(int *x) {
    (*x)++;
}

int main(void) {
    int numero = 5;
    incrementarPorReferencia(&numero);
    printf("Valor modificado: %d\n", numero); // Saída: Valor modificado: 6

    return 0;
}
```

3.4 Passagem de parâmetros por valor e por referência

É importante escolher o método apropriado com base na necessidade do programa.

A passagem por valor é geralmente mais segura e simples, enquanto a passagem por referência pode ser mais eficiente para evitar cópias desnecessárias de grandes conjuntos de dados.



3.5 Recursividade

Recursividade é um conceito em programação onde uma função chama a si mesma para resolver um problema.

Em outras palavras, é a prática de resolver um problema dividindo-o em subproblemas menores, onde a solução para cada subproblema é obtida chamando a mesma função recursivamente.



3.5 Recursividade

Na recursividade, a função continua chamando a si mesma até atingir um **caso base**, que é a condição em que a função para de se chamar recursivamente e retorna um resultado direto.

A ideia fundamental é decompor um problema complexo em subproblemas mais simples e resolver cada subproblema até chegar a uma solução final.



3.5 Recursividade

Na linguagem C, uma função pode chamar outra função.

A função main() pode chamar qualquer função, seja ela da biblioteca da linguagem (como a função printf()) ou definida pelo programador (função imprime()).

Uma função também pode chamar a si própria, a qual chamamos de função recursiva.



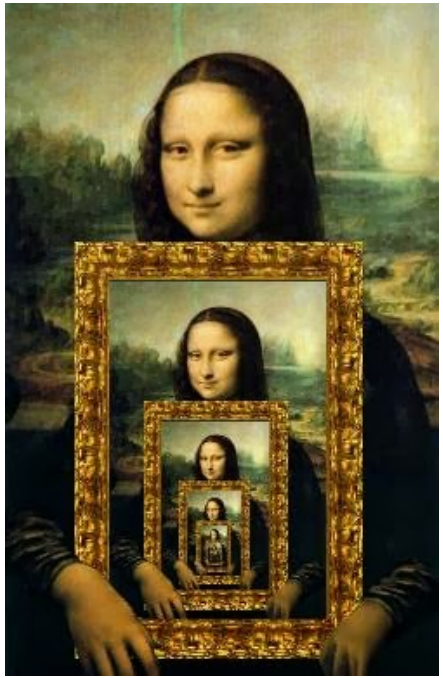
3.5 Recursividade

É importante entender que a linguagem C suporta recursividade, e você pode escrever funções recursivas da mesma forma que em outras linguagens de programação.

No entanto, é crucial garantir que a recursividade tenha um caso base para evitar que a função entre em um loop infinito.



3.5 Recursividade









3.5 Recursividade

Um processo recursivo para a solução de um problema consiste em duas partes:

1) O caso base, cuja solução é conhecida;
2) Um método geral que reduz o problema a um ou mais problemas menores (subproblemas) de mesma natureza.

Muitas funções podem ser definidas recursivamente. Para isso, é preciso identificar as duas partes acima.

3.5 Recursividade

Exemplos clássicos de recursividade: **Fatorial** de um número e o n-ésimo termo da sequência de **Fibonacci**.

Fatorial:
- 3! = 3 * 2!
- 4! = 4 * 3!
- n! = n * (n - 1)!

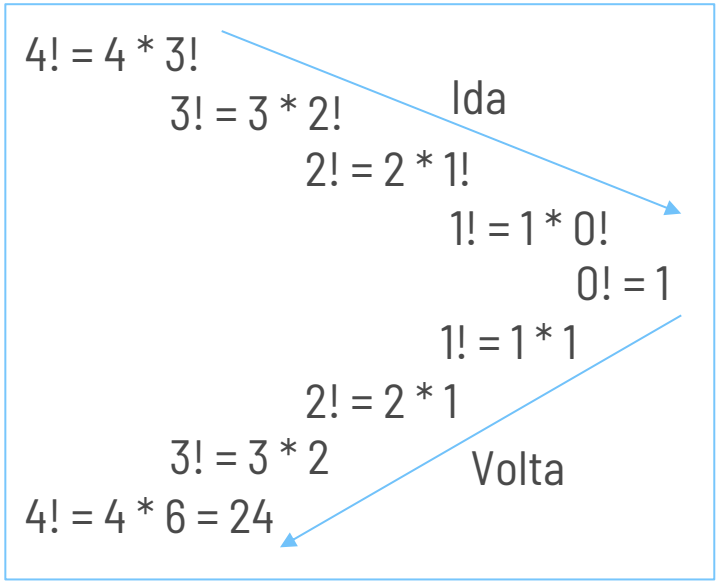
Fibonacci:
0,1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, ...
Fn = Fn-1 + Fn-2

3.5 Recursividade

A recursão também é chamada de definição circular. Ela ocorre quando algo é definido em termos de si mesmo. Vejamos no caso do Fatorial de um número:

0! = 1
1! = 1 * 0!
2! = 2 * 1!
3! = 3 * 2!
4! = 4 * 3!

n! = n * (n – 1)! : fórmula geral
0! = 1 : caso-base



3.5 Recursividade

Fatorial de 0 até 10:

0! = 1
1! = 1
2! = 2 * 1 = 2
3! = 3 * 2 * 1 = 6
4! = 4 * 3 * 2 * 1 = 24
5! = 5 * 4 * 3 * 2 * 1 = 120
6! = 6 * 5 * 4 * 3 * 2 * 1 = 720
7! = 7 * 6 * 5 * 4 * 3 * 2 * 1 = 5040
8! = 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 40320
9! = 9 * 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 362880
10! = 10 * 9 * 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 3628800

3.5 Recursividade

Assim, usando-se este processo recursivo, o cálculo de 4!, por exemplo, é feito como a seguir:

$$
\begin{aligned}
4! &= 4 * 3! \\
&= 4 * (3 * 2!) \\
&= 4 * (3 * (2 * 1!)) \\
&= 4 * (3 * (2 * (1 * 0!))) \\
&= 4 * (3 * (2 * (1 * 1))) \\
&= 4 * (3 * (2 * 1)) \\
&= 4 * (3 * 2) \\
&= 4 * 6 \\
&= 24
\end{aligned}
$$

3.5 Recursividade

Aqui está um exemplo simples de uma função recursiva em C que calcula o fatorial de um número. Vejamos ela em três versões 1) Iterativa 2) Recursiva e 3) Recursiva em uma linha.

1

```c
int fatorialComLoop(int n) {
    int resultado = 1;
    // Utilizando um loop for para multiplicar os números de 1 a n
    for (int i = 1; i <= n; i++) {
        resultado *= i;
    }
    return resultado;
}
```

2

```c
// Função recursiva padrão
int fatorialRecursivo(int n) {
    // Caso base
    if (n == 0 || n == 1) {
        return 1;
    } else {
        // Chamada recursiva
        return n * fatorialRecursivo(n - 1);
    }
}
```

3.5 Recursividade

Aqui está um exemplo simples de uma função recursiva em C que calcula o fatorial de um número. Vejamos ela em três versões 1) Iterativa 2) Recursiva e 3) Recursiva em uma linha.

2

```c
// Função recursiva padrão
int fatorialRecursivo(int n) {
    // Caso base
    if (n == 0 || n == 1) {
        return 1;
    } else {
        // Chamada recursiva
        return n * fatorialRecursivo(n - 1);
    }
}
```



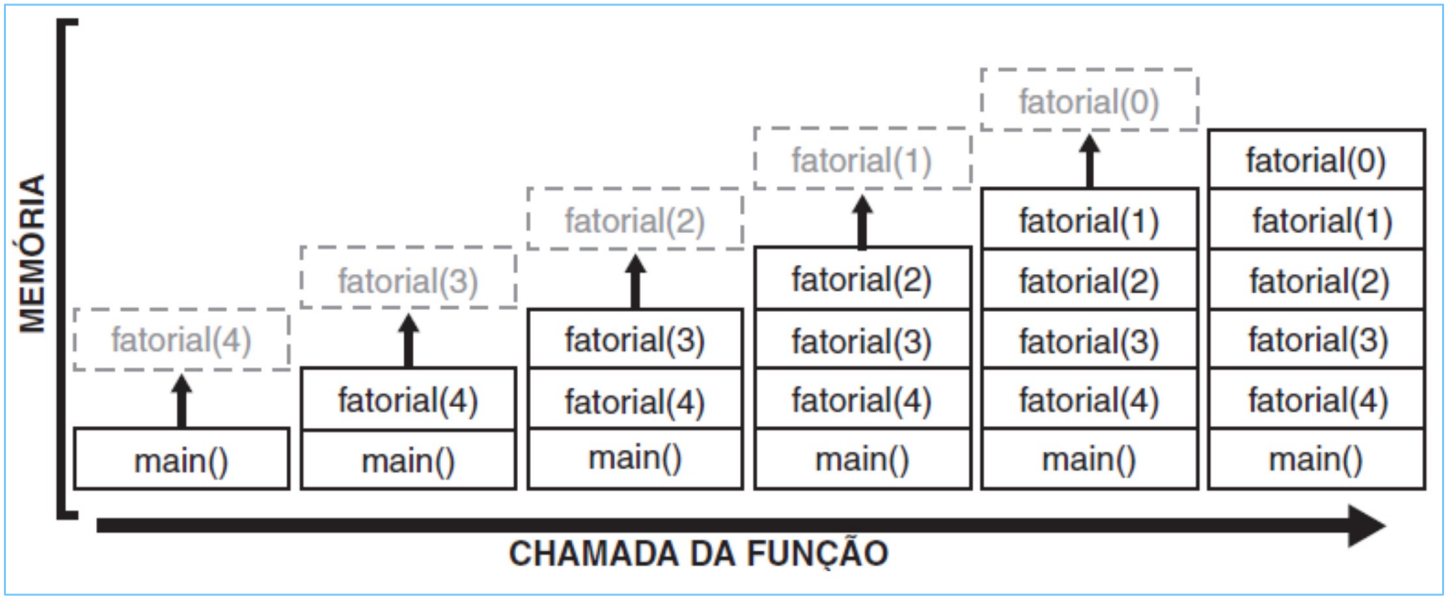
3

```c
// Função recursiva em uma linha utilizando o operador ternário
int fatorialRecursivoUmaLinha(int c) {
    return (c == 0 || c == 1) ? 1 : (c * fatorialRecursivoUmaLinha(c - 1));
}
```

3.5 Recursividade

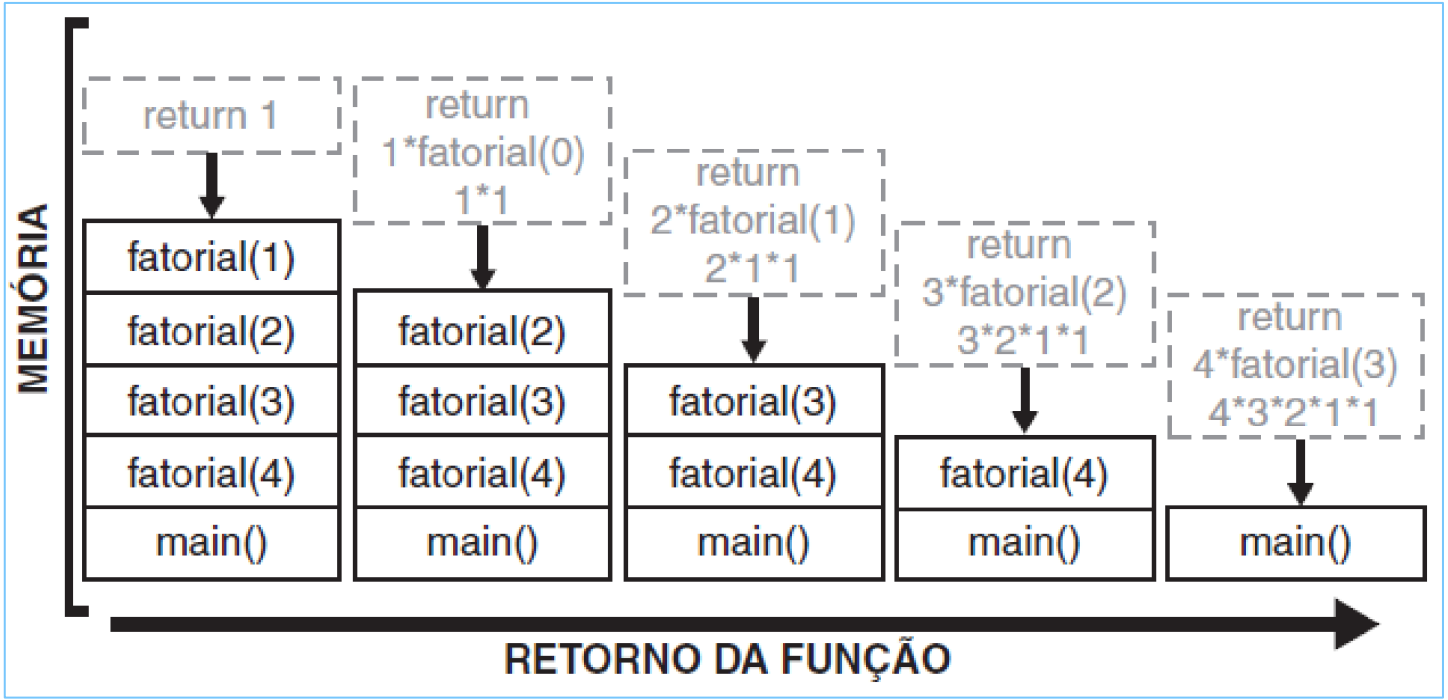
O que acontece na chamada da função fatorial com um valor como n = 4?

– int x = fatorial (4);



3.5 Recursividade

Uma vez que chegamos ao caso-base, é hora de fazer o caminho de volta da recursão.



3.5 Recursividade

Em geral, formulações recursivas de algoritmos são frequentemente consideradas "mais enxutas" ou "mais elegantes" do que formulações iterativas.

Porém, algoritmos recursivos tendem a necessitar de mais espaço na memória do que algoritmos iterativos (são menos performáticos em geral).

3.5 Recursividade

Todo cuidado é pouco ao se fazer funções recursivas.

– Critério de parada: determina quando a função deverá parar de chamar a si mesma.

– O parâmetro da chamada recursiva deve ser sempre modificado, de forma que a recursão chegue a um término.

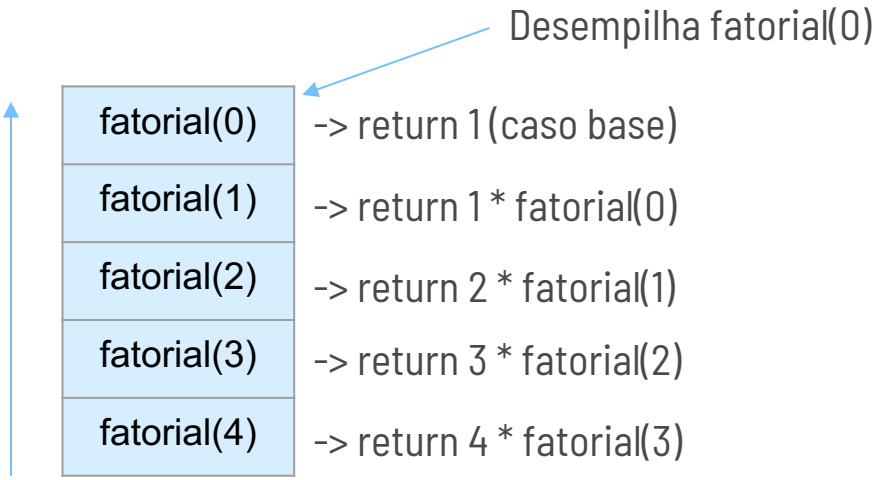
3.5 Recursividade

Mas como uma função recursiva é de fato implementada no computador?

Usando-se o mecanismo conhecido como Pilha de Execução.

Considere, novamente, o exemplo para 4!:

Pilha de Execução:



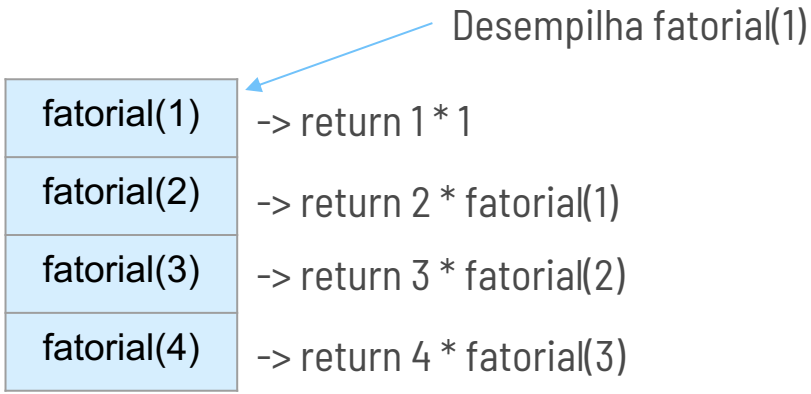
3.5 Recursividade

Mas como uma função recursiva é de fato implementada no computador?

Usando-se o mecanismo conhecido como Pilha de Execução.

Considere, novamente, o exemplo para 4!:

Pilha de Execução:



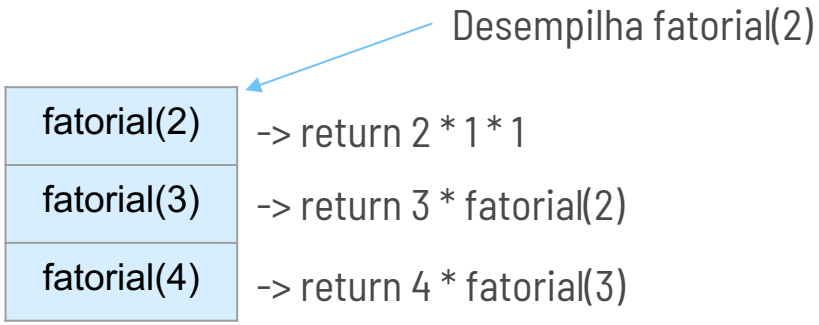
3.5 Recursividade

Mas como uma função recursiva é de fato implementada no computador?

Usando-se o mecanismo conhecido como Pilha de Execução.

Considere, novamente, o exemplo para 4!:

Pilha de Execução:



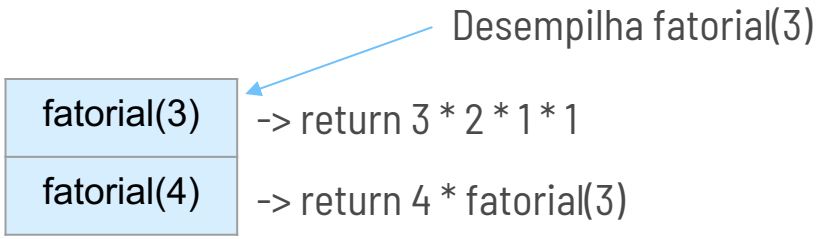
3.5 Recursividade

Mas como uma função recursiva é de fato implementada no computador?

Usando-se o mecanismo conhecido como Pilha de Execução.

Considere, novamente, o exemplo para 4!:

Pilha de Execução:



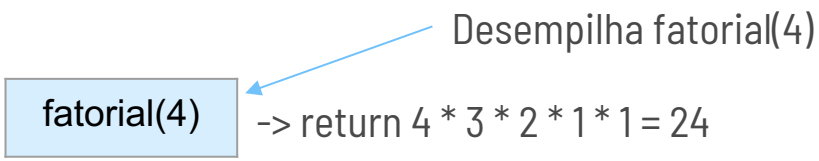
3.5 Recursividade

Mas como uma função recursiva é de fato implementada no computador?

Usando-se o mecanismo conhecido como Pilha de Execução.

Considere, novamente, o exemplo para 4!:

Pilha de Execução:



3.5 Recursividade

Vejamos agora a sequência de **Fibonacci**:

A sequência de Fibonacci é uma sequência de números inteiros em que cada número é a soma dos dois anteriores. Geralmente, a sequência começa com 0 e 1.

A sequência de Fibonacci é definida pela seguinte relação de recorrência: $F(n)=F(n-1)+F(n-2)$ onde $F(0) = 0$ e $F(1) = 1$ são os dois primeiros termos da sequência.

3.5 Recursividade

A sequência de Fibonacci começa assim: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... e assim por diante. Cada número na sequência é chamado de "número de Fibonacci". A sequência continua indefinidamente, gerando uma série de números que exibe uma variedade de propriedades interessantes e relações matemáticas.

A sequência de Fibonacci tem várias aplicações em matemática, ciência da computação, biologia e arte.

3.5 Recursividade

Ela também aparece em diversos fenômenos naturais, como padrões de crescimento em plantas e a disposição de folhas em algumas plantas.

Além disso, a sequência de Fibonacci está relacionada ao chamado "número de ouro" (ou proporção áurea), que é uma constante matemática associada à estética e à harmonia em várias disciplinas.

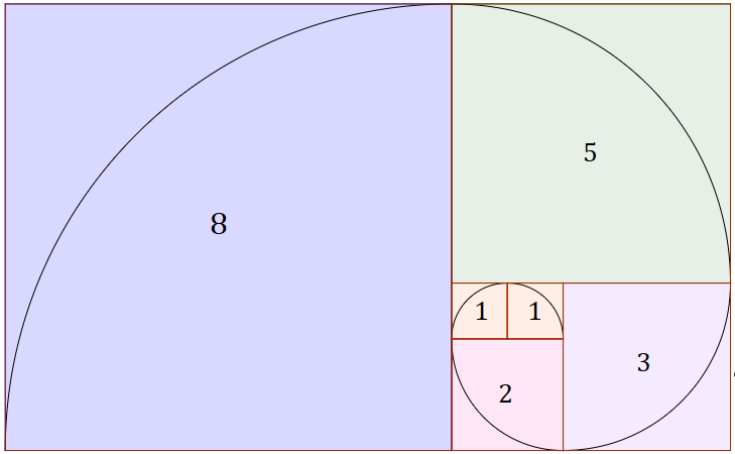
3.5 Recursividade

Leonardo de Pisa, também conhecido como Fibonacci, foi um matemático italiano do século XIII. Ele é mais conhecido por popularizar a sequência de Fibonacci, mas seu trabalho se estende a várias áreas da matemática.

A palavra "Fibonacci" foi cunhada por historiadores muito depois de sua morte, como uma abreviação de "filius Bonacci", que significa "filho de Bonacci", uma referência ao pai de Leonardo.

3.5 Recursividade

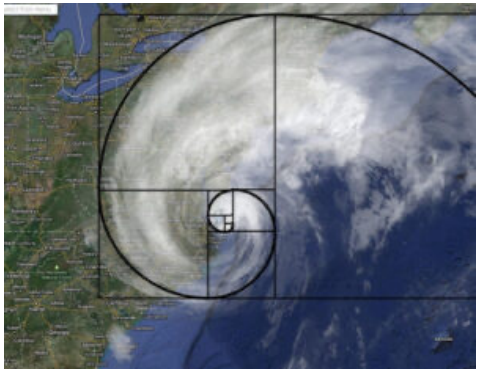
Fibonacci introduziu a sequência de Fibonacci em seu livro "Liber Abaci" ("O Livro do Cálculo"), publicado em 1202. A sequência estava originalmente relacionada a um problema prático sobre o crescimento de uma população de coelhos.



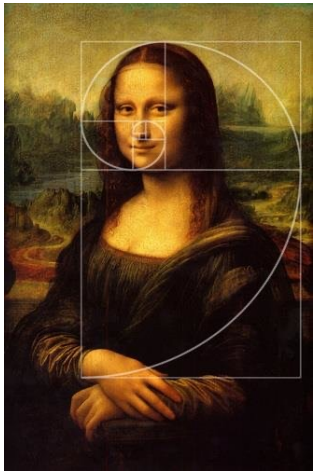


3.5 Recursividade



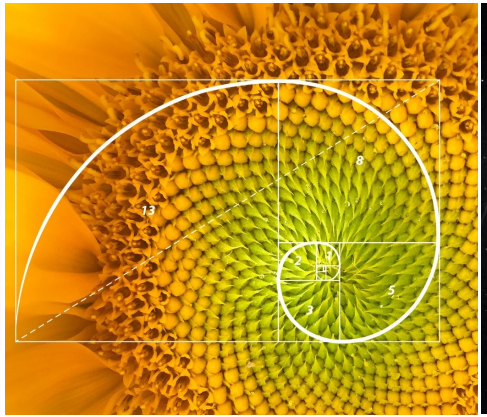


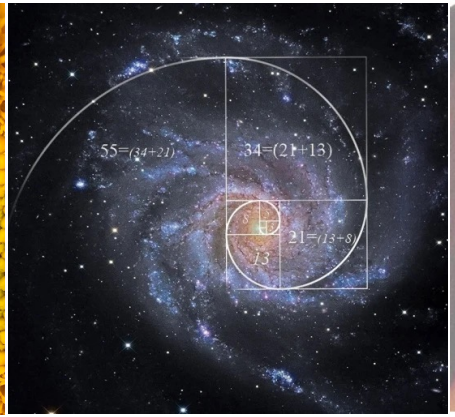


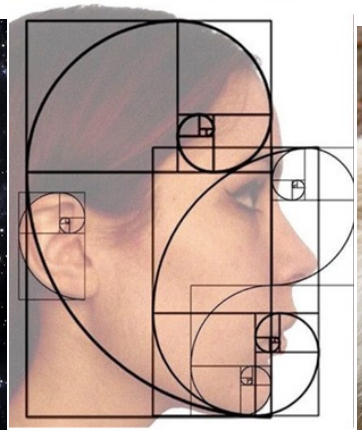








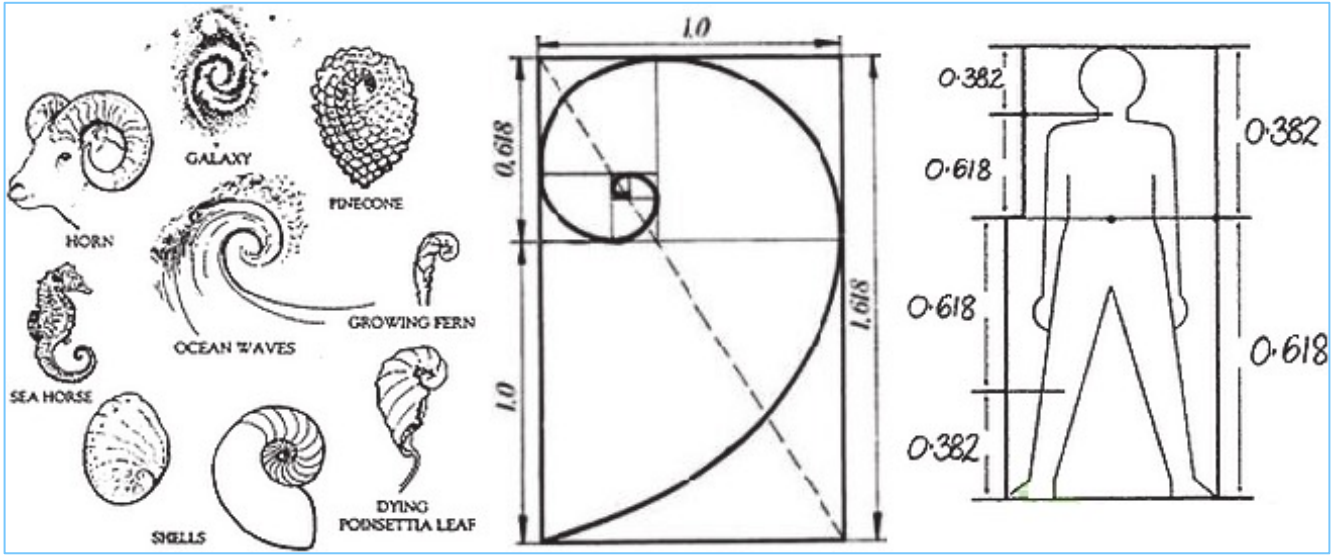






3.5 Recursividade

Leitura sugerida: aqui



3.5 Recursividade

Essa sequência é um clássico da recursão:

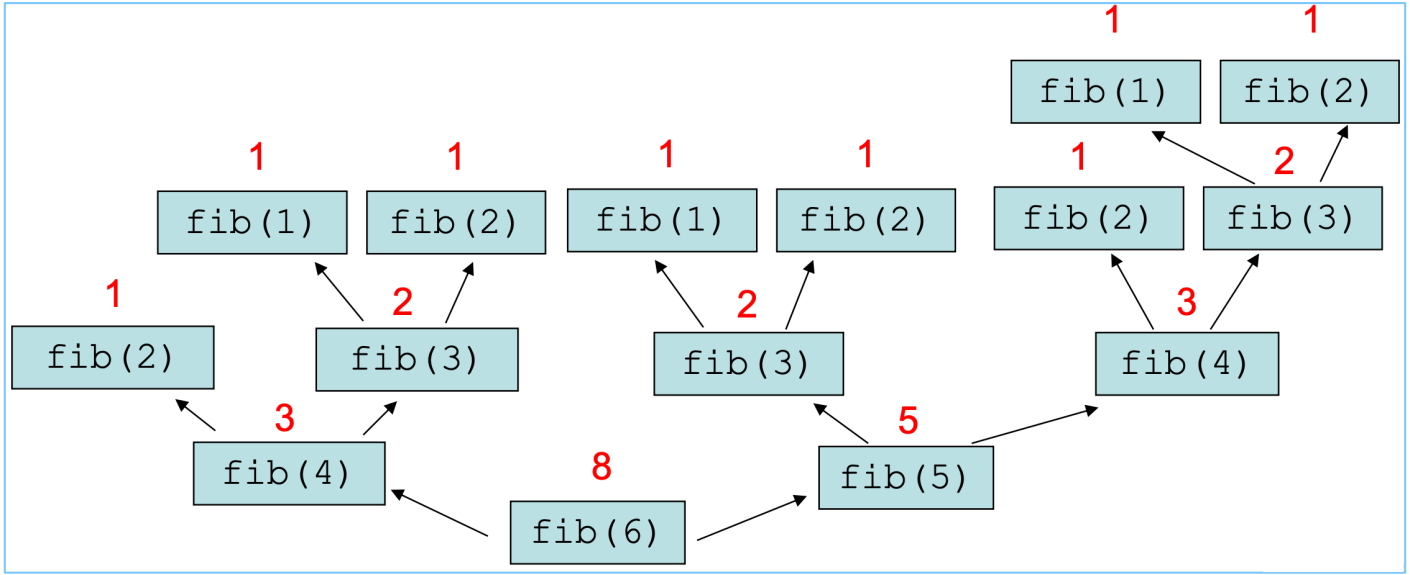
– 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

Sua solução recursiva é muito elegante:

```
int fibo(int n){
if (n == 0 || n == 1){
    return n;
} else
    return fibo(n-1) + fibo(n-2);
}
```

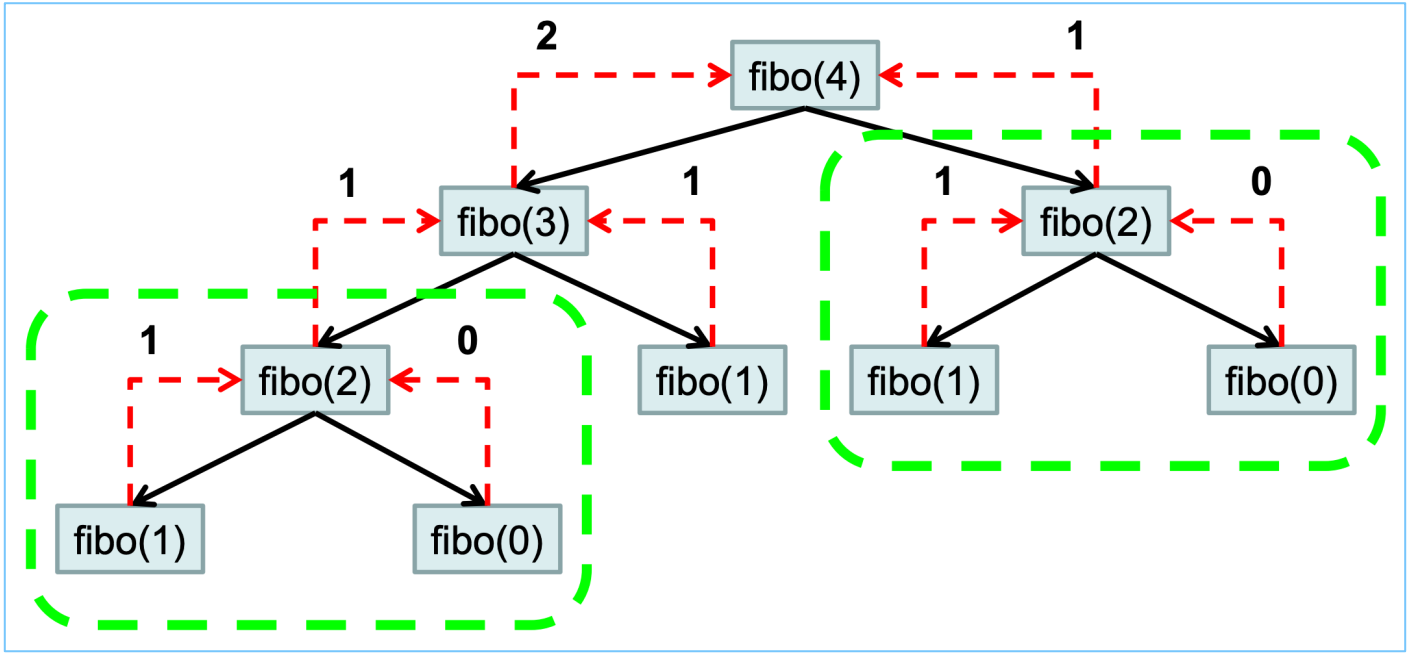
3.5 Recursividade

A função recursiva para o cálculo do n-ésimo termo da sequência de fibonacci é extremamente ineficiente, uma vez que recalcula o mesmo valor várias vezes.



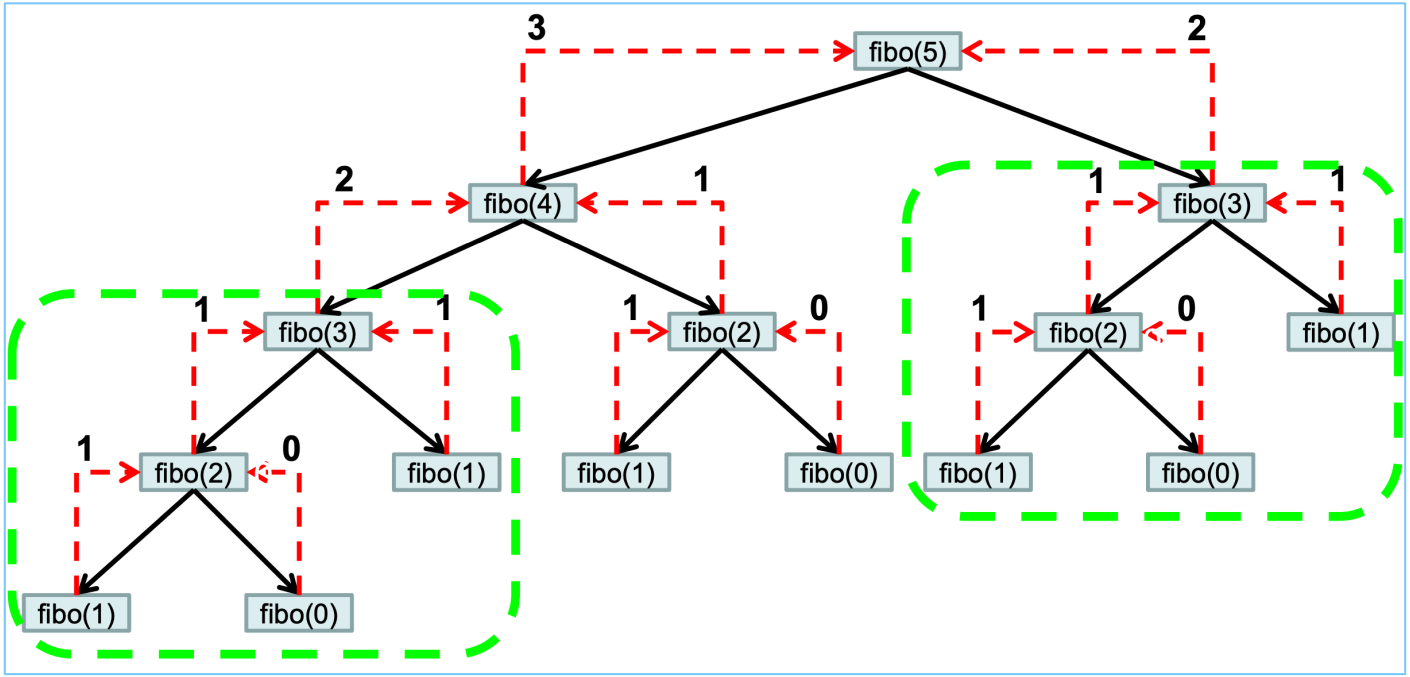
3.5 Recursividade

Como se verifica na imagem, elegância não significa eficiência:



3.5 Recursividade

Aumentando para fibo(5):



3.5 Recursividade

Quando você chama fib(n), ela se divide em duas chamadas recursivas fib(n-1) e fib(n-2). No entanto, se você observar mais de perto, perceberá que muitos dos subproblemas são recalculados várias vezes.

Isso ocorre porque a recursividade não armazena os resultados intermediários, resultando em uma quantidade significativa de repetição de trabalho.

3.5 Recursividade

Por exemplo, se calcularmos fib(5), a função calculará fib(4) e fib(3). Quando calculamos fib(4), ele também calculará fib(3) e fib(2), e assim por diante.

Você verá que fib(3) é recalculado várias vezes, levando a um aumento exponencial no número total de chamadas recursivas e operações repetidas.

3.5 Recursividade

A complexidade de tempo dessa abordagem é exponencial (O(2^n)), o que a torna impraticável para valores relativamente pequenos de n. Para resolver esse problema de eficiência, é comum usar abordagens mais eficientes, como a programação dinâmica ou a técnica de memoização, para armazenar resultados intermediários e evitar recálculos desnecessários.

Essas técnicas reduzem a complexidade de tempo para O(n) ou O(n log n), tornando o algoritmo muito mais rápido para valores grandes de n.

3.5 Recursividade

Aqui está um exemplo de uma função em C que encontra o n-ésimo termo da sequência de **Fibonacci**. Vejamos ela em três versões 1) Iterativa, 2) Recursiva e 3) Recursiva em uma linha.

1

```c
//Utiliza um loop para calcular os termos da sequência de Fibonacci
int fiboComLoop(int n) {
    if (n == 0 || n == 1) {
        return n;
    } else {
        int a = 0, b = 1, resultado;

        for (int i = 2; i <= n; ++i) {
            resultado = a + b;
            a = b;
            b = resultado;
        }
        return resultado;
    }
}
```

3.5 Recursividade

Função recursiva em C que encontra o n-ésimo termo da sequência de **Fibonacci**.

2

```c
// Função recursiva padrão
int fiboRecursivo(int n) {
    if (n == 0 || n == 1) {
        return n;
    } else
        return fiboRecursivo(n - 1) + fiboRecursivo(n - 2);
}
```

3

```c
// Função recursiva em uma linha utilizando o operador ternário
int fiboRecursivoUmaLinha(int n) {
    return (n == 0) ? 0 : ((n == 1) ? 1 : (fiboRecursivoUmaLinha(n - 1) + fiboRecursivoUmaLinha(n - 2)));
}
```

3.5 Recursividade

O livro dos autores Brassard e Bradley (Fundamentals of Algorithmics, 1996, pág. 73) apresenta um quadro comparativo de tempos de execução das versões iterativa e recursiva:

n	10	20	30	50	100
recursivo	8 ms	1 s	2 min	21 dias	10^9 anos
iterativo	0.17 ms	0.33 ms	0.50 ms	0.75 ms	1,50 ms

Portanto: um algoritmo recursivo nem sempre é o melhor caminho para se resolver um problema. No entanto, a recursividade muitas vezes torna o algoritmo mais simples.

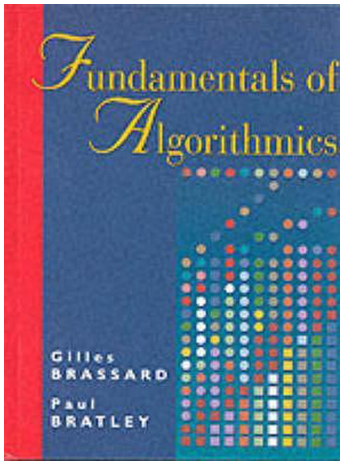
3.5 Recursividade

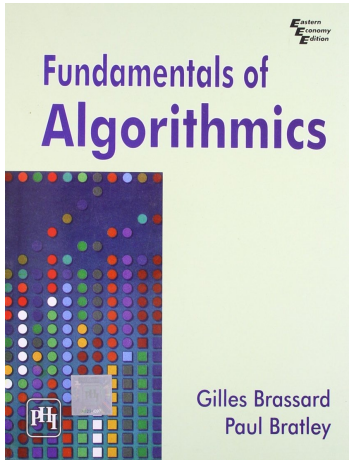
Referências:

https://dcc.ufmg.br/professor/mirella-moura-moro/

https://homepages.dcc.ufmg.br/~mirella/DCC003/Slides/Aula11-Recursividade.pdf

https://www.amazon.com.br/Fundamentals-Algorithmics-Gilles-Bassard/dp/0133350681





3.5 Recursividade

Veja também:

Recursividade:

[Versão 2010/2011] https://github.com/joaopauloaramuni/c/blob/main/Aramuni.c
[Versão 2024] https://github.com/joaopauloaramuni/algoritmos-e-estruturas-de-dados-i/blob/main/PROJETOS/Prj_Aramuni_Recursivo/src/Prj_Aramuni_Recursivo.c

Linguagem C:

[Versão 2010/2011] https://github.com/joaopauloaramuni/c
[Versão 2024] https://github.com/joaopauloaramuni/algoritmos-e-estruturas-de-dados-i

*2010/2011 foram os anos em que cursei as disciplinas de Estrutura de Dados I e II, ministradas pelo professor Rafael Nunes Papa.

3.5 Recursividade

```c
int palindrome(char str[100], int tam) {
    return (tam <= 1) || (*str == str[tam - 1] && palindrome(str + 1, tam - 2));
}

int strLen(char *orig, int cont) {
    return (*orig++) ? cont++, strLen(orig, cont) : cont;
}

void strCpy(char *str1, char *str2) {
    (*str1) ? *str2 = *str1, str1++, str2++, strCpy(str1, str2) : 0;
}

void strCat(char *str1, char *str2, int p) {
    (*str2) ? str1[p] = *str2, p++, str2++, strCat(str1, str2, p) : 0;
}

void strInverte(char *str1, char *str2, int p, int i, int j) {
    (p != 0) ? (str1++, p--, strInverte(str1, str2, p, i, j)) : (i++, (j++ != i) ? (i--, *str2 = *str1, str1--, str2++, strInverte(str1, str2, p, i, j)) : 0);
}

void strLimpa(char *str1) {
    (*str1) ? *str1 = ' ', str1++, strLimpa(str1) : 0;
}

int contLetra(char palavra[100], int cont, char letra, int i) {
    return (palavra[i] == letra) ? contLetra(palavra, ++cont, letra, ++i) : (palavra[i] ? contLetra(palavra, cont, letra, ++i) : cont);
}

int fatorial(int c) {
    return (c == 0 || c == 1) ? 1 : (c * fatorial(c - 1));
}

int fibonacci(int n) {
    return (n == 0) ? 0 : ((n == 1) ? 1 : (fibonacci(n - 1) + fibonacci(n - 2)));
}

int mdc(int a, int b) {
    return (a == 0) ? b : ((b == 0) ? a : mdc(b, a % b));
}

int powExpo(int base, int expo) {
    return (expo == 0) ? 1 : (expo == 1 ? base : (base * powExpo(base, expo - 1)));
}

void hanoi(int disc, char orig, char dest, char aux) {
    (disc != 1) ? A, C, B : C;
}
```

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:

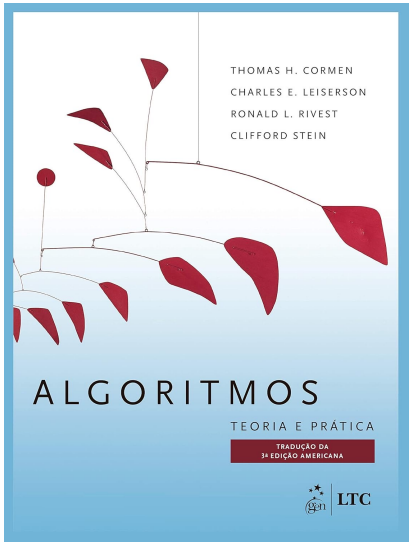
- CORMEN, Thomas. **Algoritmos** - Teoria e Prática. Editora GEN LTC. 3ª edição, 2012.

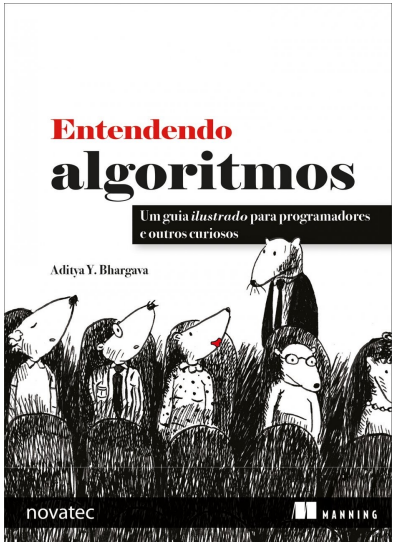
- BHARGAVA, Aditya Y. **Entendendo Algoritmos**: Um Guia Ilustrado Para Programadores e Outros Curiosos. Novatec Editora, 2017.

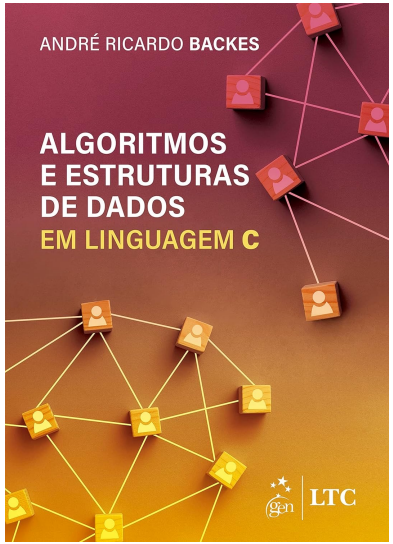
- BACKES, André Ricardo. **Algoritmos e Estruturas de Dados em Linguagem C**. Editora LTC, 2022.

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:







Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:

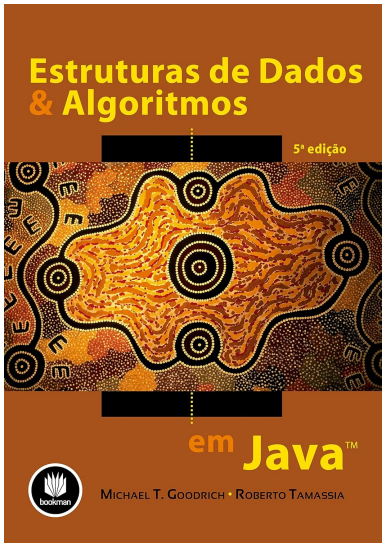
- GOODRICH, Michael T. **Estruturas de Dados e Algoritmos em Java**. Editora Bookman. 5ª edição, 2013.

- AGARWAL, Basant. **Estruturas de Dados e Algoritmos com Python**: Armazene, manipule e acesse dados de forma eficaz e melhore o desempenho de suas aplicações. Novatec Editora, 2023.

- GRONER, Loiane. **Estruturas de Dados e Algoritmos com JavaScript**: Escreva um Código JavaScript Complexo e Eficaz Usando a Mais Recente ECMAScript. Novatec Editora, 2019.

Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:







Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:

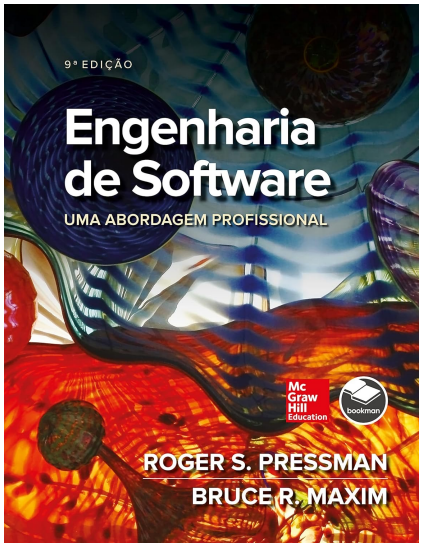
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**. Editora Grupo AMGH, 2021.

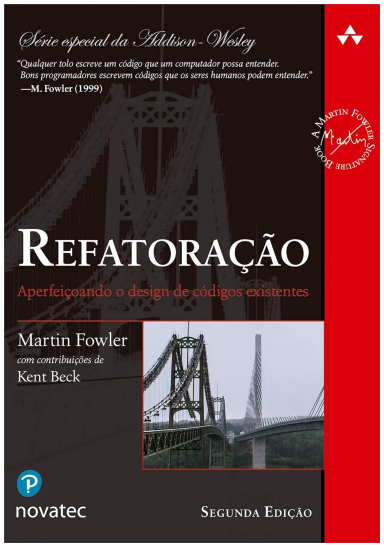
- FOWLER, Martin. **Refatoração**: Aperfeiçoando o Design de Códigos Existentes. Editora Novatec, 2020.

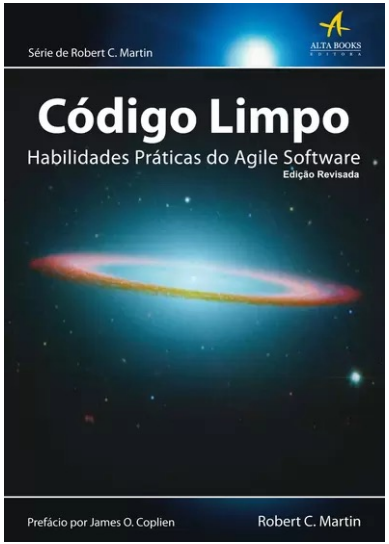
- MARTIN, Robert C. **Código limpo**: habilidades práticas do Agile software. Editora Alta Books, 2009.

Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:







Disciplina – Algoritmos e Estruturas de Dados I

Nesta disciplina usaremos: Linguagem C (ANSI Style)

- Linguagem C
[DevDocs] https://devdocs.io/c/

- Eclipse IDE
[Download] https://www.eclipse.org/downloads/packages/release/2023-12/r/eclipse-ide-cc-developers
[Docs] https://www.eclipse.org/documentation/





Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes